

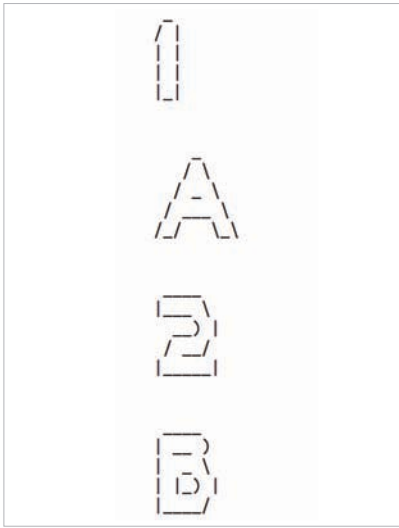


Figure 5: Output of the Python script using asyncio

concurrent and parallel programming, it is important to note that the former allows a program to make progress on multiple tasks simultaneously but not necessarily at the same time. On the other hand, parallel programming refers to executing multiple tasks simultaneously, typically on different processing units or cores to complete the tasks faster than sequential execution.

Asynchronous programming is a specific approach to concurrent programming that allows a program to make progress on multiple tasks without blocking or waiting for the completion of each task. `asyncio` comes to our aid here. `asyncio` is a Python library that facilitates concurrent programming for building asynchronous network applications, and it can also be applied in machine learning and AI applications. For instance, if a large data set requires multiple machine learning models to be trained on different parts of the data, `asyncio` can execute each model's training algorithm asynchronously on separate parts of the data set, reducing the overall training time for the models. However, it is important to optimise the code and ensure that the asynchronous execution does not introduce errors. To install the library `asyncio`, execute

the command `pip3 install asyncio` in the terminal. The following Python script demonstrates the use of `asyncio` (line numbers are included for ease of explanation).

```
1. import asyncio
2. from art import text2art
3. async def print_number( ):
4.     for i in range(1, 3):
5.         print(text2art(f"{i}"))
6.         await asyncio.sleep(1)
7.     async def print_letter( ):
8.         alphabet = ['A', 'B']
9.         for letter in alphabet:
10.            print(text2art(f"{letter}"))
11.            await asyncio.sleep(1)
12.     async def main( ):
13.         await asyncio.gather(print_
            number(), print_letter())
14.     asyncio.run(main( ))
```

Now, let us go through the code line by line. Line 1 imports the library `asyncio`. Line 2 imports a function called `text2art()` from a library called 'art', which is used to convert plain text to ASCII art. To install the library `art`, execute the command `pip3 install art` in the terminal. It is worth mentioning that the printing of ASCII art is purely for aesthetic purposes and is not directly linked to asynchronous programming. Line 3 defines an asynchronous function called `print_number()` that will print numbers in ASCII art format. Lines 4 and 5 print the ASCII art representation of the numbers 1 and 2. Line 6 makes the function pause the execution for 1 second. Lines 7 to 11 define an asynchronous function called `print_letter()`, similar to the asynchronous function `print_number()`, but `print_letter()` will print letters instead of numbers in ASCII art format. Line 12 defines an asynchronous function

called `main()` that will call both the asynchronous functions `print_number()` and `print_letter()` concurrently. Line 13 runs both these asynchronous functions concurrently using the function `gather()` of `asyncio` and waits for them to finish execution. Finally, Line 14 runs the main function using the function `run()` of `asyncio`. Figure 5 shows the output of this Python script. It can be observed from Figure 5 that the two asynchronous functions are executed in an interleaved fashion.

Having familiarised ourselves with various Python libraries and packages used for AI, it is now time to conclude this discussion. In this article, we have explored some previously uncharted areas of AI and some very useful Python libraries. Undoubtedly, there are numerous Python libraries and packages that are valuable for creating AI applications, which we could have continued discussing in great detail. However, this abundance of options emphasises the importance of making informed choices about our tools early on in an individual's AI career.

There is still much ground to cover in the final article in this series. Hence, in the next article, we will begin by addressing the important topics that we have either omitted entirely or mentioned only briefly, and we will also provide guidance on how you can independently explore these topics. Throughout this series, our discussions have mainly relied on Python as the programming language of choice. Nonetheless, it is worth noting that Python is not the only language used for developing AI based applications. Therefore, in the final article of this series, we will provide a brief overview of the other popular programming languages used for developing AI based applications. **END** 🐧

 By: DeePU Benson

The author is a free software enthusiast and his area of interest is theoretical computer science. He maintains a technical blog.